



Uniwersytet
Wrocławski

Self-supervised Learning

Wykład szkoleniowy OAI III

Klaudia Dynak

Instytut Informatyki, Uniwersytet Wrocławski

Jak uczą się modele?

Modele samonadzorowane dla wizji

RotNet

SimCLR

BYOL

Vision Transformer – transformer dla obrazów

ViT

DINO

Jak uczą się modele?

1. Dane (uwaga! czasem nie mamy etykiet):

$$\mathcal{D} = (\mathbf{X}, \mathbf{Y}) = \{x_n, y_n\}_{n=1}^N \subseteq \mathbb{X} \times \mathbb{Y} .$$

2. Model (funkcja przekształcająca wejście w wyjście):

$$f_{\theta} : \mathbb{X} \rightarrow \mathbb{Y}; \quad f_{\theta}(x) = y .$$

3. Predykcja (wynik modelu dla konkretnej instancji danych):

$$f_{\theta}(x) = \hat{y} .$$

4. Optimalizacja funkcji straty (wybór odpowiednich parametrów modelu):

$$\theta^* = \arg \max_{\theta} \mathcal{L}(\mathcal{D}, \theta) .$$

Różne scenariusze trenowania modeli

- **Uczenie nadzorowane** (dostępne X & Y , optymalizujemy dopasowanie predykcji i etykiet, np. regresja, klasyfikacja)
- **Uczenie nienadzorowane** (dostępne tylko X , optymalizujemy własności predykcji, np. grupowanie, redukcja wymiaru)
- **Uczenie samonadzorowane** (dostępne tylko X , tworzymy syntetyczne Y z X , optymalizujemy dopasowanie predykcji i wytworzonych etykiet)
- **Uczenie półnadzorowane** (dostępne X , dostępne Y tylko dla wybranych przykładów)
- **Uczenie transferowe** (mamy wyszkolony model, odcinamy ostatnie warstwy i zastępujemy ją nową *głową*)
- **Dostrajanie** (mamy wyszkolony model, przeprowadzamy dodatkowe szkolenie wybranych warstw)

Jak wytworzyć nadzór na podstawie danych wejściowych?

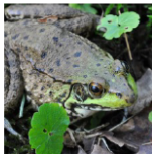
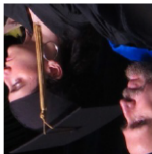
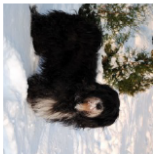
- **Zadania pretekstowe** (zmodyfikujemy dane i nauczymy model odwracać lub rozpoznawać tę modyfikację)
- **Uczenie kontrastowe** (podobne obrazki mają mieć bliskie reprezentacje, a różne — odległe; porównywanie wyjść dla zaugumentowanych danych dla jednego modelu)
- **Architektury z dwóch sieci** (wyniki dwóch siostrzanych sieci mają się stać zgodne na tym samym wejściu; porównywanie wyjść dla tych samych danych dla wielu modeli)

Modele samonadzorowane dla wizji

Modele samonadzorowane dla wizji

RotNet

RotNet – klasyfikujemy obroty



Jakie obroty zastosowano na poniższych obrazkach?

RotNet – klasyfikujemy obroty

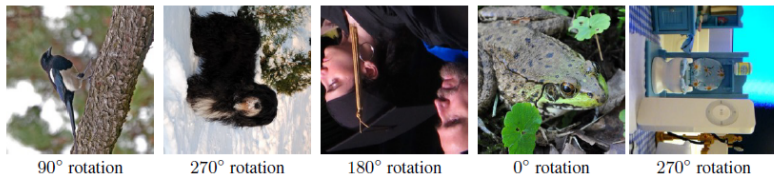


Figure 1: Images rotated by random multiples of 90 degrees (e.g., 0, 90, 180, or 270 degrees). The core intuition of our self-supervised feature learning approach is that if someone is not aware of the concepts of the objects depicted in the images, he cannot recognize the rotation that was applied to them.

<https://arxiv.org/abs/1803.07728>

RotNet – klasyfikujemy obroty

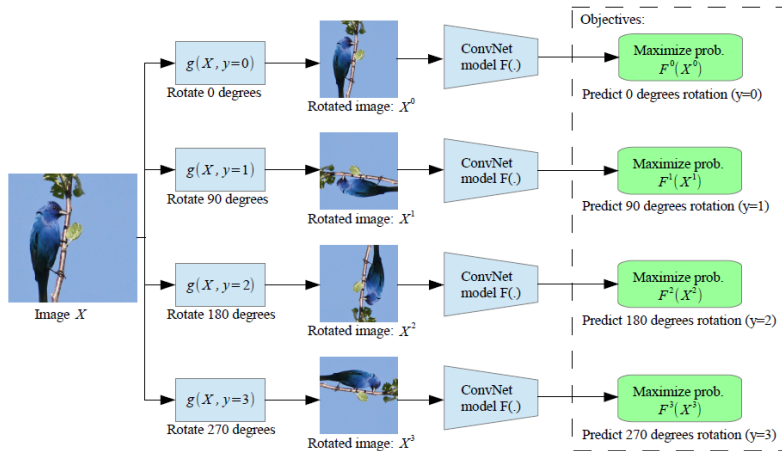


Figure 2: Illustration of the self-supervised task that we propose for semantic feature learning. Given four possible geometric transformations, the 0, 90, 180, and 270 degrees rotations, we train a ConvNet model $F(.)$ to recognize the rotation that is applied to the image that it gets as input. $F^y(X^{y*})$ is the probability of rotation transformation y predicted by model $F(.)$ when it gets as input an image that has been transformed by the rotation transformation y^* .

- kolorowanie obrazka (<https://arxiv.org/abs/1603.08511>)

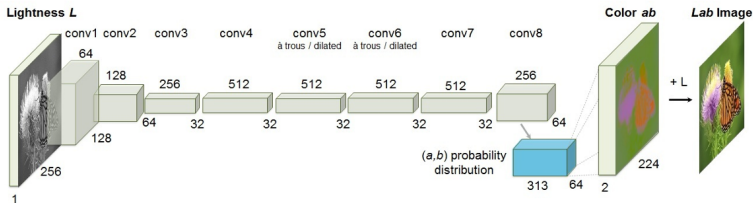


Fig. 2. Our network architecture. Each **conv** layer refers to a block of 2 or 3 repeated **conv** and **ReLU** layers, followed by a **BatchNorm** [30] layer. The net has no **pool** layers. All changes in resolution are achieved through spatial downsampling or upsampling between **conv** blocks.

- puzzle Jigsaw (<https://arxiv.org/abs/1603.09246>)

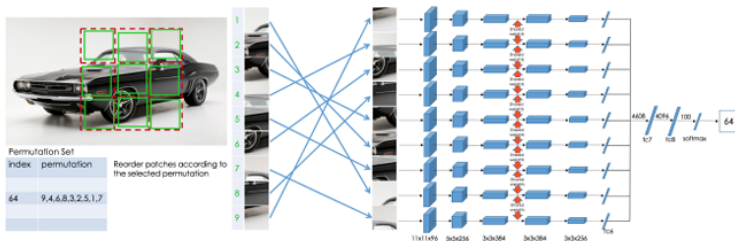


Fig. 3: **Context Free Network.** The figure illustrates how a puzzle is generated and solved. We randomly crop a 225×225 pixel window from an image (red dashed box), divide it into a 3×3 grid, and randomly pick a 64×64 pixel tiles from each 75×75 pixel cell. These 9 tiles are reordered via a randomly chosen permutation from a predefined permutation set and are then fed to the CFN. The task is to predict the index of the chosen permutation (technically, we define as output a probability vector with 1 at the 64-th location and 0 elsewhere). The CFN is a siamese-ennet CNN. For simplicity, we do not indicate the max-pooling and ReLU layers. These shared layers are implemented exactly as in AlexNet [25]. **In the transfer learning experiments we show results with the trained weights transferred on AlexNet (precisely, stride 4 on the first layer). The training in the transfer learning experiment is the same as in the other permutation methods. Notice instead that during**

Modele samonadzorowane dla wizji

SimCLR

Podobieństwo w przestrzeni zanurzeń – kosinusowe

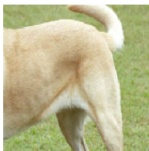
Dlaczego rozwiązanie zapewniające wyłącznie warunek "podobne blisko siebie" może być zdegenerowane?

SimCLR – podobne blisko siebie, różne – daleko

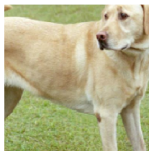
- Tylko skąd wiadomo, co jest do siebie podobne bez etykiet?
- Stwórzmy podobne obrazki!



(a) Original



(b) Crop and resize



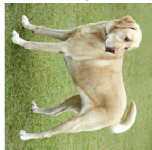
(c) Crop, resize (and flip)



(d) Color distort. (drop)



(e) Color distort. (jitter)



(f) Rotate $\{90^\circ, 180^\circ, 270^\circ\}$



(g) Cutout



(h) Gaussian noise

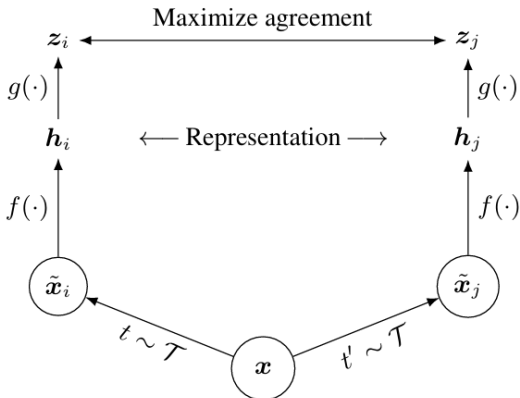


(i) Gaussian blur



(j) Sobel filtering

SimCLR – podobne blisko siebie, różne – daleko



<https://dl.acm.org/doi/10.5555/3524938.3525087>

SimCLR – podobne blisko siebie, różne – daleko

- kontrastowa funkcja straty:

$$\ell = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} 1_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau)},$$

$$\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N \ell(2k-1, 2k) + \ell(2k, 2k-1)$$

gdzie sim jest podobieństwem cosinusowym, a jako przykłady negatywne traktujemy oba widoki wszystkich pozostałych obrazów;

- stochastyczne augmentowanie danych t :
 - losowe kadrowanie (cropping),
 - losowe zniekształcenia kolorów,
 - szum gaussowski,
 - najskuteczniejsze: crop + silna zmiana kolorów w jednym widoku;
- koder $f(\cdot)$ – ResNet (<https://arxiv.org/abs/1512.03385>),
głowica projekcyjna $g(\cdot)$ – MLP;
- ogromna wielkość batcha – od 256 do 8192, optymalizator LARS,
globalna normalizacja partii.

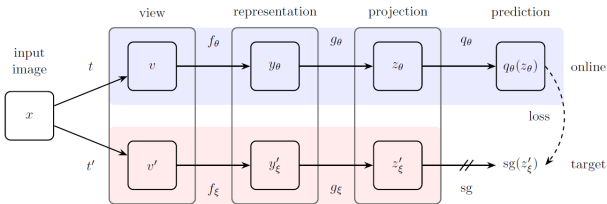
Modele samonadzorowane dla wizji

BYOL

BYOL – sieci mają dawać możliwie jak najbliższe reprezentacje

Bootstrap your own Latent

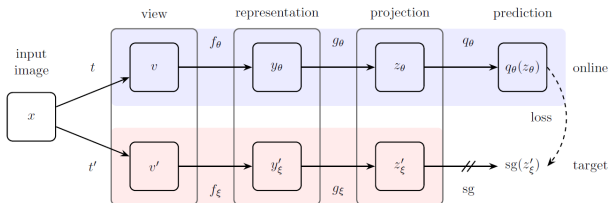
Idea: *bootstrapujemy* reprezentacje ukryte z sieci *online* i *target* – to są nasze widoki na te same dane, które chcemy do siebie zbliżyć. Dzięki zróżnicowanym widokom z sieci *target*, nie potrzebujemy już przykładów negatywnych.



Sieci *online* i *target* mają te same architektury, ale różne parametry.

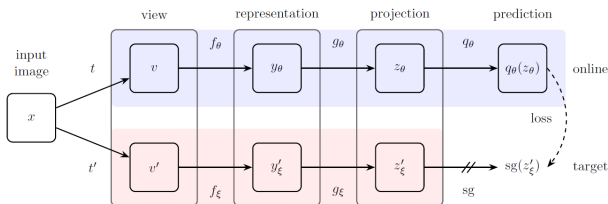
Throughout this paper, the term bootstrap is used in its idiomatic sense rather than the statistical sense.

BYOL – sieci mają dawać możliwie jak najbliższe reprezentacje



- robimy dwie różne augmentacje danych dla dwóch sieci: *online* i *target*,
- żeby zmapować wyniki obu sieci używamy modułu predykującego q_θ ,
- próbujemy przewidzieć reprezentację, którą da sieć *target*, za pomocą sieci *online*,
- całość jest mniej wrażliwa na wybór augmentacji i wielkość batcha w porównaniu do SimCLR

BYOL – sieci mają dawać możliwie jak najbliższe reprezentacje

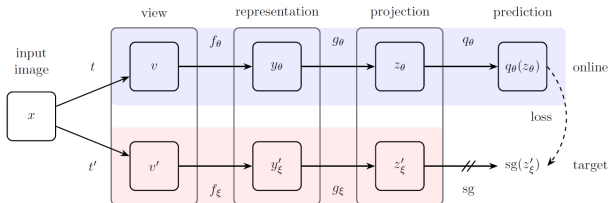


- sieć *online* uczymy po gradiencie

$$\mathcal{L}_{\theta,\xi} \triangleq \|\overline{q_\theta}(z_\theta) - \bar{z}'_\xi\|_2^2 = 2 - 2 \cdot \frac{\langle q_\theta(z_\theta), z'_\xi \rangle}{\|q_\theta(z_\theta)\|_2 \cdot \|z'_\xi\|_2}$$

optymalizowana funkcja straty to suma $\mathcal{L}_{\theta,\xi}$ dla obu przypisań
wybranych augmentacji do obu sieci (v online + v' target
vs v' online + v target)

BYOL – sieci mają dawać możliwie jak najbliższe reprezentacje



- natomiast sieć *target* małymi krokami zbliża się do sieci *online* poprzez aktualizowanie jej wag małą domieszką wag *online*

$$\xi \leftarrow \tau \xi + (1 - \tau) \theta$$

(Wykładnicza średnia krocząca – Exponential Moving Average)

Średnia krocząca:

$$a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8, a_9, a_{10}, a_{11}, a_{12}, a_{13}, \dots$$

Wykładnicza średnia krocząca (EMA):

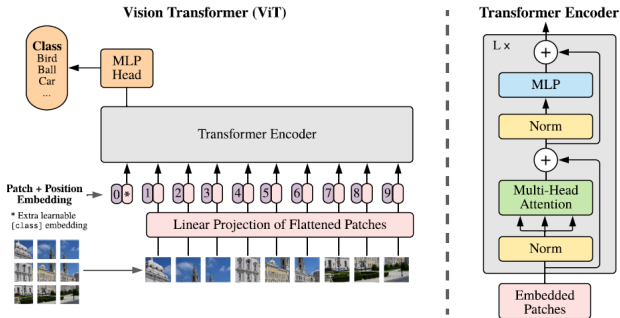
$$a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8, a_9, a_{10}, a_{11}, a_{12}, a_{13}, \dots$$

Vision Transformer – transformer dla obrazów

Vision Transformer – transformer dla obrazów

ViT

Fragmenty obrazów są traktowane tak samo jak tokeny (słowa) w przetwarzaniu języka.



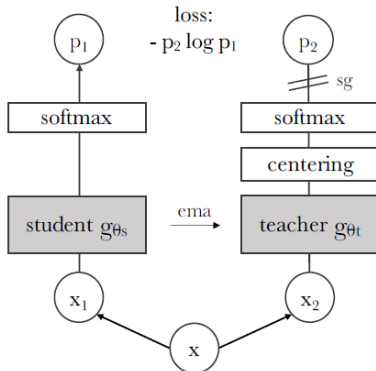
fragmenty obrazków (patche), projekcje (zanurzenia patchy), zanurzenia pozycyjne, transformer, głowica MLP

<https://openreview.net/pdf?id=YicbFdNTTy>

Vision Transformer (ViT)	CNN (e.g., ResNet)
globalne rozumienie obrazów	sąsiedztwo pikseli uwzględnione w architekturze, niewrażliwość na translację istotnych elementów obrazka
skuteczniejszy na dużych danych (14M - 300M)	wystarczą małe lub średniej wielkości dane

Vision Transformer – transformer dla obrazów

DINO

Self-Distillation with **NO** labels

- Jednym z czynników sukcesu Transformers w NLP było samonadzorowane wstępne szkolenie; oryginalnt ViT jest szkolony w sposób nadzorowany – a co jeśli dla obrazów też zastosujemy uczenie samonadzorowane?



Figure 1: Self-attention from a Vision Transformer with 8×8 patches trained with no supervision. We look at the self-attention of the [CLS] token on the heads of the last layer. This token is not attached to any label nor supervision. These maps show that the model automatically learns class-specific features leading to unsupervised object segmentations.

- na samonadzorowanych embeddingach z ViT dostajemy świetne wyniki na prostych klasyfikatorach, jak kNN i klasyfikatory liniowe (nawet 78.3% top-1 dokładności na ImageNet).

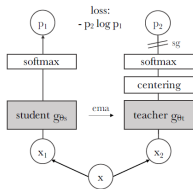
<https://arxiv.org/abs/2104.14294>

Mała sieć (uczeń) uczy się naśladować wyniki dużej, wytrenowanej sieci (nauczyciela).

Czym to się różni od uczenia się etykiet z oryginalnych danych?

Mamy dodatkową wiedzę, zakodowaną w nauczycielu (np. prawdopodobieństwa klas oddają ich podobieństwa).

DINO – destylacja wiedzy bez etykiet

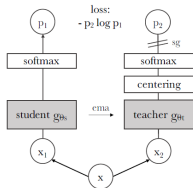


- Augmentacje **globalne** (np. obejmujące ponad 50% obrazu),
augmentacje **lokalne** (np. obejmujące mniej niż 50% obrazu),
 V — zbiór wszystkich widoków; nauczyciel ma dostęp do globalnych, uczeń do wszystkich.
- Ucznia trenujemy po gradiencie:

$$\min_{\theta_s} \sum_{x \in \{x_1^g, x_2^g\}} \sum_{\substack{x' \in V \\ x' \neq x}} H(P_t(x), P_s(x'))$$

minimalizując entropię krzyżową.

DINO – destylacja wiedzy bez etykiet



- Obie sieci mają tę samą architekturę, ale różne parametry.
- Nauczyciel nabywa wiedzę w trakcie uczenia się całego modelu (EMA):

$$\theta_t \leftarrow \lambda \theta_t + (1 - \lambda) \theta_s,$$

λ wg schematu cosinusowego od 0.996 do 1.0.

- Żeby uniknąć zdegenerowania sieci, używamy centrowania nauczyciela i skalowania temperaturą.

Chen, T., Kornblith, S., Norouzi, M. & Hinton, G.. (2020). A Simple Framework for Contrastive Learning of Visual Representations. *Proceedings of the 37th International Conference on Machine Learning*, in *Proceedings of Machine Learning Research* 119:1597-1607 Available from <https://proceedings.mlr.press/v119/chen20j.html>.

Grill, J. B., Strub, F., Altché, F., Tallec, C., Richemond, P., Buchatskaya, E., ... & Valko, M. (2020). Bootstrap your own latent-a new approach to self-supervised learning. *Advances in neural information processing systems*, 33, 21271-21284.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., ... Houlsby, N. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*.

Caron, M., Touvron, H., Misra, I., Jégou, H., Mairal, J., Bojanowski, P., & Joulin, A. (2021). Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 9650-9660).



Uniwersytet
Wrocławski

Self-supervised Learning

Wykład szkoleniowy OAI III

Klaudia Dynak

Instytut Informatyki, Uniwersytet Wrocławski